3 Dimensional array explained

A NumPy 3D array, also known as a 3-dimensional ndarray, is a data structure used in the NumPy library for efficient numerical computation in Python. It can be visualized as a collection of 2D arrays (matrices) stacked on top of each other, forming a cuboid or a volume.

- **Key characteristics:**

Dimensions (Axes):

A 3D array has three axes:

First axis (depth/slices): Represents the number of 2D arrays or "slices" in the stack.

Second axis (rows): Represents the number of rows within each 2D array.

Third axis (columns): Represents the number of columns within each row of each 2D array.

Homogeneous Data Type:

All elements within a NumPy array, including 3D arrays, must be of the same data type (e.g., all integers, all floats).

Fixed Size:

Once created, the total number of elements in a NumPy array cannot

change, although its shape can be reconfigured using methods like reshape().

Rectangular Shape:

Each "layer" or "slice" in a 3D array must have the same number of rows and columns, ensuring a consistent, non-"jagged" structure.

Creation and Indexing:

A 3D array can be created using np.array() by nesting lists to represent the dimensions.

Example:

```
import numpy as np

arr = np.array([
    [[1, 2, 3, 4],
     [5, 6, 7, 8],
     [9, 10, 11, 12]],

    [[13, 14, 15, 16],
```

```
    [17, 18, 19, 20],
    [21, 22, 23, 24]]
])
print(arr)
print(arr.shape) # Output: (2, 3, 4)
```

Elements are accessed using multiple indices, corresponding to each dimension:

```
# Access the element in the first slice, second row, third column (value 7)
element = arr[0, 1, 2]
print(element) # Output: 7
```

The most easiest way. We all have studied "Sets". Just consider 3D

numpy array as the formation of "sets".

x = np.zeros((2,3,4))

Simply Means:

2 Sets, 3 Rows per Set, 4 Columns

Example:

x = np.zeros((2,3,4))

Output

Set # 1 ---- [[[0., 0., 0., 0.], ---- Row 1

[0., 0., 0., 0.], ---- Row 2

[0., 0., 0., 0.]], ---- Row 3

Set # 2 ---- [[0., 0., 0., 0.], ---- Row 1

[0., 0., 0., 0.], ---- Row 2

[0., 0., 0., 0.]]] ---- Row 3

numpy.array() function

The numpy.array() function is used to create an array. This function takes an iterable object as input and returns a new NumPy array with a specified data type (if provided) and shape.

The array() function is useful when working with data that can be converted into an array, such as a list of numbers. It is often used to create a NumPy array from an existing Python list or tuple.

Syntax:

numpy.array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0)

NumPy array: array() function

Parameters:

object	An array, any object exposing the array interface, an object whose __array__ method returns an array, or any (nested) sequence.
	Required

dtype	The desired data-type for the array. If not given, then the type will be determined as the minimum type required to hold the objects in the sequence. This argument can only be used to 'upcast' the array. For downcasting, use the .astype(t) method. **optional**
copy	If true (default), then the object is copied. Otherwise, a copy will only be made if __array__ returns a copy, if obj is a nested sequence, or if a copy is needed to satisfy any of the other requirements (dtype, order, etc.). **optional**
order	Specify the memory layout of the array. If object is not an array, the newly created array will be in C order (row major) unless 'F' is specified, in which case it will be in Fortran order (column major). If object is an array the following holds.
subok	If True, then sub-classes will be passed-through, otherwise the returned array will be forced to be a base-class array (default) **optional**

ndmin	Specifies the minimum number of dimensions that the resulting array should have. Ones will be pre-pended to the shape as needed to meet this requirement **optional**

NumPy Array Copy vs View

The Difference Between Copy and View

The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.

The copy owns the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.

The view does not own the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

COPY:

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()

arr[0] = 42

print(arr)

print(x)
```

VIEW:

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.view()

arr[0] = 42

print(arr)

print(x)
```

Check if Array Owns its Data

Copies owns the data, and views does not own the data

Every NumPy array has the attribute base that returns None if the array owns the data.

Otherwise, the base attribute refers to the original object.

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()

y = arr.view()

print(x.base)

print(y.base)
```

Example

Check if the returned array is a copy or a view:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

print(arr.reshape(2, 4).base)
```

order	**nocopy**	**copy=True**
'K'	unchanged	F & C order preserved, otherwise most similar order
'A'	unchanged	F order if input is F and not C, otherwise C order
'C'	C order	C order
'F'	F order	F order

When copy=False and a copy is made for other reasons, the result is the same as if copy=True, with some exceptions for A,

The default order is ‘K’.

- C Is Contiguous layout. Mathematically speaking**, row major**.
- F Is Fortran contiguous layout., **column major**.
- A Is any order. Generally don't use this.
- K Is keep order. Generally don't use this.

NumPy Array Shape

The shape of an array is the number of elements in each dimension.

NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.

Example

Print the shape of a 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

ndmin

The ndmin argument in NumPy allows you to specify the minimum number of dimensions for an array when creating it. This means that even if you provide a 1D or 2D array as input, the resulting array will have at least the specified number of dimensions. If the input array already has more dimensions than ndmin, it will not be changed.

Example

Create an array with 5 dimensions using ndmin using a vector with values 1,2,3,4 and verify that last dimension has value 4:

```
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('shape of array :', arr.shape)
```

Integers at every index tells about the number of elements the corresponding dimension has.

NumPy Array Reshaping

Reshaping means changing the shape of an array.

The shape of an array is the number of elements in each dimension.

By reshaping we can add or remove dimensions or change number of elements in each dimension.

Reshape From 1-D to 2-D

Example

Convert the following 1-D array with 12 elements into a 2-D array.

The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr.reshape(4, 3)

print(newarr)
```

Reshape From 1-D to 3-D

Example

Convert the following 1-D array with 12 elements into a 3-D array.

The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
```

```
newarr = arr.reshape(2, 3, 2)

print(newarr)
```

We Reshape Into any Shape, as long as the elements required for reshaping are equal in both shapes.

We can reshape an 8 elements 1D array into 4 elements in 2 rows 2D array but we cannot reshape it into a 3 elements 3 rows 2D array as that would require 3x3 = 9 elements.

Flattening the arrays

Flattening array means converting a multidimensional array into a 1D array.

.reshape(-1)

Example

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

NumPy - Iterating Over Array

Using a for Loop

Example

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
for element in arr:
  print(element)
```

Example: Iterating Over a 2D Array

```
import numpy as np
# Create a 2D NumPy array (3x3 matrix)
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
for row in arr_2d:
  print(row)
```

- **Using 'nditer' Iterator Object**

The nditer() function in NumPy provides an efficient multidimensional iterator object that can be used to iterate over elements of arrays. It uses Python's standard iterator interface to visit each element of an array.

Example

```
import numpy as np

# Example array

arr = np.array([[1, 2], [3, 4]])

# Iterate using nditer

for x in np.nditer(arr):

  print(x)
```

Array Operations

Operations on array

Example

a = np.array([[12,53,25,75],[4,6,8,10],[9,7,5,3],[47,92,13,71]], dtype=int)

array([[12, 53, 25, 75],

[4, 6, 8, 10],

[9, 7, 5, 3],

[47, 92, 13, 71]])

- **Changing the dimensions of the array:**

a.reshape(8,2)

array([[12, 53],

[25, 75],

[4, 6],

[8, 10],

[9, 7],

[5, 3],

[47, 92],

[13, 71]])

Example 2

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])

reshaped_array = np.reshape(a, (3, 2), order='F')

print(a)

print(reshaped_array)
```

- **Convert array into 1D:**

```
a.ravel()

array([12, 53, 25, 75,  4,  6,  8, 10,  9,  7,  5,  3, 47, 92, 13, 71])
```

- **Array of unique elements:**

```
np.unique(a)
```

array([3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 25, 47, 53, 71, 75, 92])

- **Minimum and maximum value of the array**:

print(a.min(),a.max())

3 92

- **Sum of all elements, takes optional argument axis 1 for row wise addition 0 for column wise addition:**

a.sum()

440

a.sum(axis=1)

array([165, 28, 24, 223])

- **To find the Standard Deviation:**

np.std(a)

28.902854530305479

To find the square root of elements:

```
np.sqrt(a)

array([[ 3.46410162,  7.28010989,  5.        ,  8.66025404],
       [ 2.        ,  2.44948974,  2.82842712,  3.16227766],
       [ 3.        ,  2.64575131,  2.23606798,  1.73205081],
       [ 6.8556546 ,  9.59166305,  3.60555128,  8.42614977]])
```

Operations on 2 numpy arrays.

```
a = np.array([[1,2,3],[4,5,6]])

b = np.array([[10,11,1],[13,14,1]])
```

a+b

```
array([[11, 13,  4],
       [17, 19,  7]])
```

a*b

```
array([[10, 22,  3],
       [52, 70,  6]])
```

a/b

```
array([[ 0.1       ,  0.18181818,  3.        ],
       [ 0.30769231,  0.35714286,  6.        ]])
```

a-b

```
array([[-9, -9,  2],
       [-9, -9,  5]])
```

```
b = np.array([[10,11],[13,14],[1,2]])
```

a.dot(b)

```
import numpy as np

a = np.array([[1,2],[3,4]])
b = np.array([[11,12],[13,14]])
x= a.dot(b)
print(x)
```

Other Operations

pad() Returns a padded array with shape increased according to pad_width

Example

```
import numpy as np

array = np.array([1, 2, 3])

padded_array = np.pad(array, pad_width=2, mode='constant')

print("Padded Array:", padded_array)
```

```
Padded Array: [0 0 1 2 3 0 0]
```

- **Transpose Operations**

Transpose Permutes the dimensions of an array

Example

Transpose operation switching the rows and columns of the 2D array −

```
import numpy as np

# Original 2D array

array_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Transposing the 2D array

transposed_2d = np.transpose(array_2d)

print("Original array:\n", array_2d)
```

```
print("Transposed array:\n", transposed_2d)
```

- **Joining Arrays**

  concatenate Joins a sequence of arrays along an existing axis

Example 1

```
import numpy as np

# Define two 1-D arrays

array1 = np.array([1, 2, 3])

array2 = np.array([4, 5, 6])

# Concatenate along the default axis (axis=0)

result = np.concatenate((array1, array2))

print("Result of concatenation:", result)
```

Example 2

```
import numpy as np

# Define two 2-D arrays

array1 = np.array([[1, 2], [3, 4]])

array2 = np.array([[5, 6], [7, 8]])

# Concatenate along axis 0 (rows)
```

```
result = np.concatenate((array1, array2), axis=0)

print("Result of concatenation along axis 0:", result)
```

Example 3

```
import numpy as np

# Define two 2-D arrays

array1 = np.array([[1, 2], [3, 4]])

array2 = np.array([[5, 6], [7, 8]])

# Concatenate along axis 1 (columns)

result = np.concatenate((array1, array2), axis=1)

print("Result of concatenation along axis 1:", result)
```

- **Splitting Arrays**

  split Splits an array into multiple sub-arrays

Syntax

```
numpy.split(ary, indices_or_sections, axis=0)
```

Example

```
import numpy as np

# Create an array

arr = np.arange(9)

print(arr)

# Split the array into 3 equal parts

result = np.split(arr, 3)

print("\nSplit array into 3 equal parts:")

for i, sub_array in enumerate(result):

    print(f"Sub-array {i+1}:")

    print(sub_array)
```

- **Adding / Removing Elements**

  resize Returns a new array with the specified shape

Syntax

```
numpy.resize(arr, shape)
```

Example

```
import numpy as np

# Create a 2D array

a = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)

# Resize the array to shape (3, 2)

b = np.resize(a, (3, 2))

print(b)

print('\n')

print(b.shape)

# Resize the array to shape (3, 3)

# Note: This will repeat elements of 'a' to fill the new shape

print('Resize the second array:')

b = np.resize(a, (3, 3))

print(b)
```

- **append** Appends the values to the end of an array

Syntax

```
numpy.append(arr, values, axis)
```

Example

```
import numpy as np

a = np.array([[1,2,3],[4,5,6]])

print(np.append(a, [7,8,9]))

print('Append elements along axis 0:')

print(np.append(a, [[7,8,9]],axis = 0))

print(\n'Append elements along axis 1:')

print(np.append(a, [[5,5,5],[7,8,9]],axis = 1))
```

- **insert** Inserts the values along the given axis before the given indices

Syntax

```
numpy.insert(arr, obj, values, axis = None)
```

Example

```
import numpy as np

# Define the initial array

a = np.array([[1, 2], [3, 4], [5, 6]])

print(np.insert(a, 3, [11, 12]))
```

- **delete** Returns a new array with sub-arrays along an axis deleted

Syntax

Numpy.delete(arr, obj, axis)

Example

```
import numpy as np

# Creating an array with shape (3, 4)

a = np.arange(12).reshape(3, 4)

# Delete the element at index 5 from the flattened array

print('Array flattened before delete operation as axis not used:')

print(np.delete(a, 5))
```

- **Rearranging Elements**

  flip() Reverse the order of elements in an array along the given axis

Syntax

numpy.flip(m, axis=None)

Example

```
import numpy as np

my_array = np.array([10, 20, 30, 40, 50])

print("Original Array:", my_array)

result = np.flip(my_array)
```

```
print("Reversed Array:", result)
```

- **Sorting**

  sort() Return a sorted copy of an array

Syntax

```
numpy.sort(a, axis=-1, kind=None, order=None)
```

Example

```
import numpy as np

a = np.array([25, 5, 15, 10])

a = np.sort(a)

print("Original Array:", a)

print("Sorted Array:", b)
```

- **argmax()** Returns the indices of the maximum values along an axis

```
import numpy as np

array = np.array([10, 20, 5, 30, 15])

max_index = np.argmax(array)

print("Array:", array)

print("Index of Maximum Value:", max_index)
```

- **argmin()** Returns the indices of the minimum values along an axis

NumPy - Indexing & Slicing

NumPy Indexing

NumPy Indexing is used to access or modify elements in an array. Three types of indexing methods are available field access, basic slicing and advanced indexing.

Example 1

```
import numpy as np

a = np.arange(10)

b = a[6]

print(b)
```

Slicing in NumPy

NumPy Slicing is an extension of Python's basic concept of slicing to n dimensions. A Python slice object is constructed by giving start, stop, and step parameters to the built-in slice function. This slice object is passed to the array to extract a part of array.

Example 1

```
import numpy as np

arr = np.arange(6)

print(arr[-2:])
```

Example 2

```
import numpy as np
arr = np.arange(12)
even_num = arr[::2]
print("Even Numbers:", even_num)
```

Example 3

```
import numpy as np
arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print(arr[:, 1])
```

Following is an output of the above code −

```
[20 50 80]
```